Multimodal Visual Understand

Function Description

After the program runs, input questions about visual content or describe the current scene through the terminal. The program will take a photo of the current environment and upload it to the Alibaba Cloud platform. Then the vision model will analyze the image with the question and provide an answer.

Startup

Open the terminal and enter the following command:

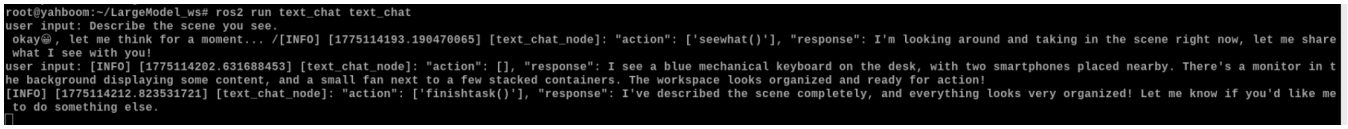
```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

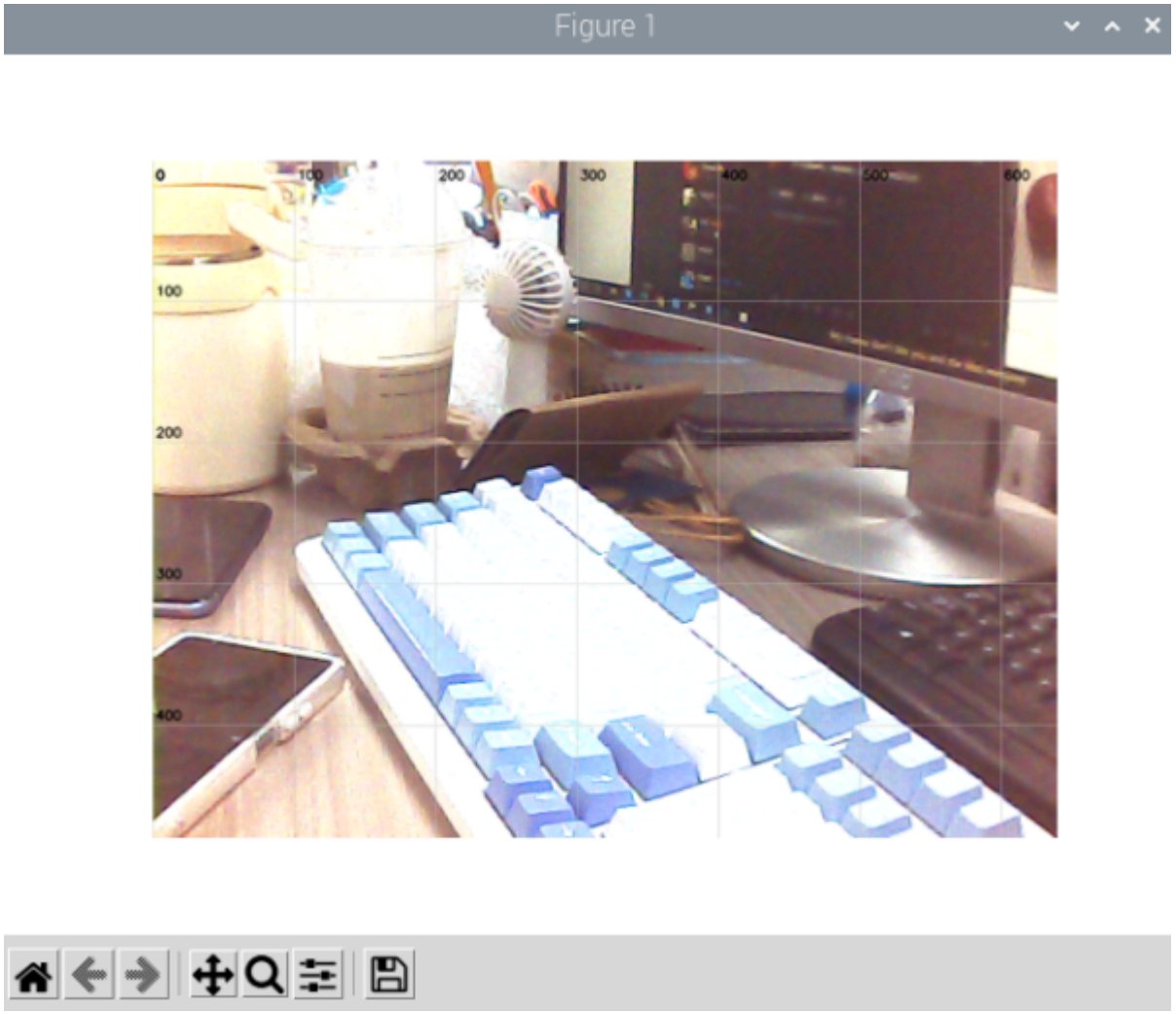
Then input questions related to visual understanding in the text_chat terminal. You can refer to the following examples:

```
Describe the scene you see.
```



The path where the captured photo is saved is:

`LargeModel_ws/install/largemodel/share/largemodel/resources_file/image.png`



Core Code Analysis

The program calls the seewhat function to capture photos. The source code path is: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def seewhat(self):
    #Save the image
    self.save_single_image()
    time.sleep(3.0)
    msg = String(data="seewhat")
    self.seewhat_handle_pub.publish(
        msg
    )  # Normalize, publish the seewhat topic, called by model_service to invoke the large
model

def save_single_image(self):
    """
    Save a single image / Save a single image
    """
    self.IS_SAVING=True
    time.sleep(0.5)
    if self.image_msg is None:
        self.get_logger().warning("No image received yet.")  # Image not yet received...
        return
```

```
    try:
        # Convert ROS image message to OpenCV image / Convert ROS image message to OpenCV
image
        cv_image = self.bridge.imgmsg_to_cv2(self.image_msg, "bgr8")
        # Save the image / Save the image
        cv2.imwrite(self.image_save_path, cv_image)
        display_thread = threading.Thread(target=self.display_saved_image)
        display_thread.start()

    except Exception as e:
        self.get_logger().error(f"Error saving image: {e}")  # Error saving image...
    self.IS_SAVING=False
```